PHONEPE · PRODUCT / BUSINESS ANALYSIS CASE STUDY

Bill-Splitting for Groups

From Problem Framing to Release-Ready Requirements

PROBLEM FRAMING | JTBD | USER STORIES | SCOPE DEFINITION

PREPARED BY YATHARTH APHALE

DECK OVERVIEW

PhonePe

What's Inside This Case Study

1	**Problem Framing**	5W1H → Problem Statement → How Might We → Jobs To Be Done → Root Cause (5 Whys).
2	**Requirements Translation**	The validated problem translated into Business, Functional, and Product requirements.
3	**Story Quality Control**	The junior PM's two draft stories audited against INVEST, then rewritten.
4	**New User Stories**	Three release-ready stories, each with Happy / Sad / Edge-Case acceptance criteria.
5	**Scope Definition**	Stakeholder alignment, in/out scope for Release 1, and the reasoning behind the line.

THE BRIEF

Case Background

PhonePe (500M+ registered users) has greenlit a group bill-splitting feature for urban users who share expenses for rent, trips, and dining. Before any design or development starts, the team needs alignment on what is being built, and why.

WHAT THE TEAM IS WALKING IN WITH

1. Three teams hold three different definitions of “bill splitting.”
2. Compliance has flagged unresolved UPI transaction-limit questions.
3. No one has written down the problem this feature solves, or for whom.
4. A junior PM started writing user stories before requirements existed.
5. Engineering wants a scope boundary before sprint planning next week.

There is energy and intent. What's missing is alignment and structure.

5W1H Investigation

WHO	Urban PhonePe users who regularly share recurring group expenses — flatmates, trip groups, dining circles (age 22–35).
WHAT	Users manually calculate shares, chase payments individually, and track who's paid — no single “who owes what” view exists.
WHEN	Recurring — every group event; peaks around trips (lump sum) and monthly rent/utilities.
WHERE	Entirely outside PhonePe — WhatsApp calculators, Splitwise, mental math — then payment via PhonePe P2P.
WHY	No native capability to split, request from many people at once, track partial payments, or reconcile.
HOW MUCH	Not yet quantified — a Problem Validation exercise must establish scale before build.

Problem Statement

Urban PhonePe users who regularly share group expenses (rent, trips, dining) face a lack of any native way to split, request, track, and settle shared costs within the app — resulting in manual calculation errors, awkward chase-ups, no visibility into who has paid, and expense-tracking happening entirely outside PhonePe — pushing users toward competitor tools for a job PhonePe is otherwise positioned to own.

SPECIFIC

Names the exact user and the exact capability gap.

MEASURABLE

Observable proxies: manual errors, chase-up friction, competitor-app usage.

IMPACT-FOCUSED

The cost isn't inconvenience — it's losing engagement to Splitwise.

How Might We

?

How might we help urban PhonePe users who share group expenses split, request, and track shared costs seamlessly within the app — so they no longer need to leave PhonePe (or use a separate app) to manage money among friends?

Discovery framing — opens the design space before requirements narrow it back down.

Jobs To Be Done

FUNCTIONAL

WHEN I've just paid for a group expense, I WANT TO instantly split the cost and request each person's share, SO I CAN recover my money without manual math or messaging.

EMOTIONAL

WHEN I'm fronting money for a group, I WANT TO feel in control — not “the annoying one” chasing people — SO I CAN preserve the relationship while getting paid back.

SOCIAL

WHEN I'm splitting bills with friends, I WANT the process to feel effortless and transparent, SO I CAN be seen as organized, not as the one creating awkwardness.

Why it matters for scoping: a pure-feature mindset only solves the functional job. Emotional and social jobs point to specific requirements — low-friction reminders, and a shared “who owes what” view visible to the group.

Root Cause — The 5 Whys

1	**Why is bill-splitting unscoped today?**	Because it was never built as a native capability.
2	**Why was it never built?**	P2P payments were designed for 1:1 transfers, not group-context tracking.
3	**Why does that matter now?**	“Splitting” fans one request to many recipients — hits UPI's transaction/day limits.
4	**Why is this a blocker?**	Collecting from many people against one “expense” risks reading as pooled funds.

ROOT CAUSE

The MVP's technical ceiling is set by UPI limits — not by design ambition.

Business, Functional & Product Requirements

BUSINESS (WHY)

Increase engagement and retention among urban users by capturing group-expense management natively in-app — reducing reliance on Splitwise.

No feature names, no UI — pure business outcome.

FUNCTIONAL (WHAT)

- Create a group expense, split among members
- Calculate each share (equal split at MVP)
- Send a request to each member
- Show real-time status: Paid / Pending
- Send reminders to pending members
- Display settlement summary

PRODUCT (HOW)

Tap “Split a Bill,” select members, enter total → app auto-calculates an equal split. “Request” fires simultaneous UPI requests (limit-aware). Once all paid, expense moves to Split History.

Junior PM Story 1 — Critique

"As a user, I want to split bills so that it is easier."

Independent	Fail	Too abstract to know what it depends on
Negotiable	Pass	Nothing concrete enough to even negotiate
Valuable	Fail	"Easier" is undefined — easier than what, for whom?
Estimable	Fail	Engineering cannot size "split bills"
Small	Fail	Could mean a calculator or a full ledger system
Testable	Fail	No way to write acceptance criteria for "easier"

ROOT PROBLEMS

"As a user" names no real persona. "So that it is easier" fails the So-That test: it restates the action rather than a genuine outcome — a vague feature, not a need.

Junior PM Story 1 — Rewrite

Split into two INVEST-compliant stories — the original bundled "split" and "understand my share" together.

STORY 1A

As a user who has just paid for a shared group expense, I want to split the total cost equally among selected group members, so that I don't have to manually calculate each person's share.

STORY 1B

As a user who has split a bill, I want to see a clear breakdown of who owes how much, so that both I and the group have shared clarity without a side conversation.

Junior PM Story 2 — Critique

"As a PhonePe user, I want the app to automatically calculate each person's share, send payment requests to all members at once, allow partial payments, send reminders, show a settlement summary, and sync with UPI history so that managing expenses is seamless."

Independent	Fail	Bundles 6 distinct capabilities that can ship independently
Negotiable	Fail	Reads as a fixed spec / wishlist, not a discussable need
Valuable	Partial	Real need is valid but buried under a feature list
Estimable	Fail	Impossible to size — 6+ epics disguised as one story
Small	Fail	Epic-sized, not Story-sized
Testable	Fail	"Seamless" can't be tested; each sub-feature needs its own AC

ROOT PROBLEM: an Epic wearing a Story's clothing — the entire feature's requirement list pasted into "I want to" format, with no prioritization or MVP/future distinction.

Junior PM Story 2 — Epic Breakdown

EPIC: GROUP BILL SPLITTING

RELEASE 1 — MVP

- Automatically calculate each person's equal share, so I don't do the math manually.
- Send a payment request to all group members at once, so I don't ask each person individually.
- See a settlement summary once everyone has paid, so I know the expense is fully closed.

RELEASE 2+

- Automated reminders to members with pending shares, so I don't personally chase anyone.
- Allow group members to pay in partial installments, so I'm not blocked by cash-flow timing.
- Sync settled expenses with my UPI transaction history, for one unified spending record.

PhonePe

New User Story 1 + Acceptance Criteria

As a user who has paid for a group dinner, I want to split the bill equally among selected friends and send payment requests in one action, so that I get repaid without individually asking each person.

Happy Path	GIVEN I've entered a total and selected 4 members WHEN I tap “Request” THEN each member receives an equal-share request and I see all 4 marked “Pending.”
Sad Path	GIVEN a selected member has no linked UPI ID WHEN I tap “Request” THEN the system excludes them, notifies me, and lets me mark their share “Settle Manually.”
Edge Case	GIVEN the total doesn't divide evenly WHEN the split is calculated THEN the remainder goes to the requester's own share, so shares sum exactly to the total.

New User Story 2 + Acceptance Criteria

As a group member who has received a bill-split request, I want to see exactly what the charge is for and pay it directly within PhonePe, so that I can settle my share without leaving the app.

Happy Path	GIVEN I receive a request WHEN I open the notification THEN I see the title, total, my share, requester, and members — and can pay in one tap.
Sad Path	GIVEN my payment fails WHEN it fails THEN I see a clear reason and my share stays “Pending” (never falsely marked Paid), so I can retry.
Edge Case	GIVEN the requester edits or cancels before I pay WHEN I open the request THEN I see an “Updated” flag, and cannot pay the outdated amount.

New User Story 3 + Acceptance Criteria

As a user managing a group expense, I want a real-time status view of who has paid and who hasn't, so that I know at a glance whether the expense is fully settled.

Happy Path	GIVEN I've sent requests to 4 members WHEN 2 pay their share THEN the card updates live to 2 Paid / 2 Pending, with a running total.
Sad Path	GIVEN all members have paid WHEN the last payment confirms THEN the expense moves to Settled, and a summary is generated in Split History.
Edge Case	GIVEN a payment is stuck in a UPI processing state WHEN I view status THEN I see a distinct “Processing” state — not Paid, not Pending.

Stakeholder Alignment Snapshot

STAKEHOLDER	STATED POSITION	REAL NEED UNDERNEATH
Product Team A	*"Bill splitting = trip settlement"*	Needs one-time, lump-sum group splits
Product Team B	*"Bill splitting = recurring rent/utilities"*	Needs recurring, repeatable group splits
Product Team C	*"Bill splitting = dining/instant splits"*	Needs fast, low-friction small-group splits
Compliance	*UPI limits unresolved for multi-recipient requests*	Flow must respect per-transaction/daily limits
Engineering	*Needs a scope boundary before sprint planning*	Needs a bounded, sequenced backlog
Junior PM	*Already writing user stories*	Needs to pause until requirements exist

In Scope — Release 1 (MVP)

- ✓ Manual creation of a one-time group expense with equal-split calculation
- ✓ Selecting group members from existing PhonePe/UPI-linked contacts
- ✓ Sending simultaneous individual payment requests to all members
- ✓ Real-time status view (Paid / Pending / Processing) per member
- ✓ Automatic settlement summary once all shares are paid
- ✓ UPI-limit-aware request handling, with manual-settlement fallback
- ✓ Basic Split History log of past settled expenses

Why these are IN: together they form the complete, testable "happy path" of split → request → track → settle.

Out of Scope — Deferred to Release 2+

Automated reminders

R1's one-tap request + status view already removes the need to manually ask.

Partial / installment payments

Adds multi-transaction reconciliation and compliance-review surface.

Unequal / custom splits

Equal split covers the highest-frequency use case validated in framing.

Automated recurring scheduling

An automation layer on an unproven core flow — validate first.

UPI history sync / reconciliation

Nice-to-have visibility, not required to split, request, or settle.

Cross-group / multi-currency

Out of scope for a domestic UPI-based MVP.

How the Scope Line Was Decided

The single test: does this capability directly serve "split → request → track → settle" for a one-time group expense?

VARIANTS → DEFERRED

Unequal splits, recurring splits, partial payments — each multiplies testing and compliance review without changing whether the core hypothesis is validated.

DEPENDENCIES → KEPT IN

UPI-limit checking, real-time status, settlement summary — without these the MVP cannot honestly be called functional.

CLOSING

Summary of the Reasoning Chain

1. Problem Framing (5W1H → Statement → JTBD)
2. Root Cause (UPI-limit architecture, not UX)
3. Requirements Translation (Business vs Functional vs Product)
4. Story Quality Control (INVEST failures → rewritten stories)
5. New Stories with full Happy / Sad / Edge-Case AC
6. Scope Boundary (drawn against the core JTBD loop)

Every requirement and scope decision in this deck traces back to the original problem statement — no step skipped.

Thank You

YATHARTH APHALE

github.com/Yatharth-2206

linkedin.com/in/yatharth-aphale-338b203b8